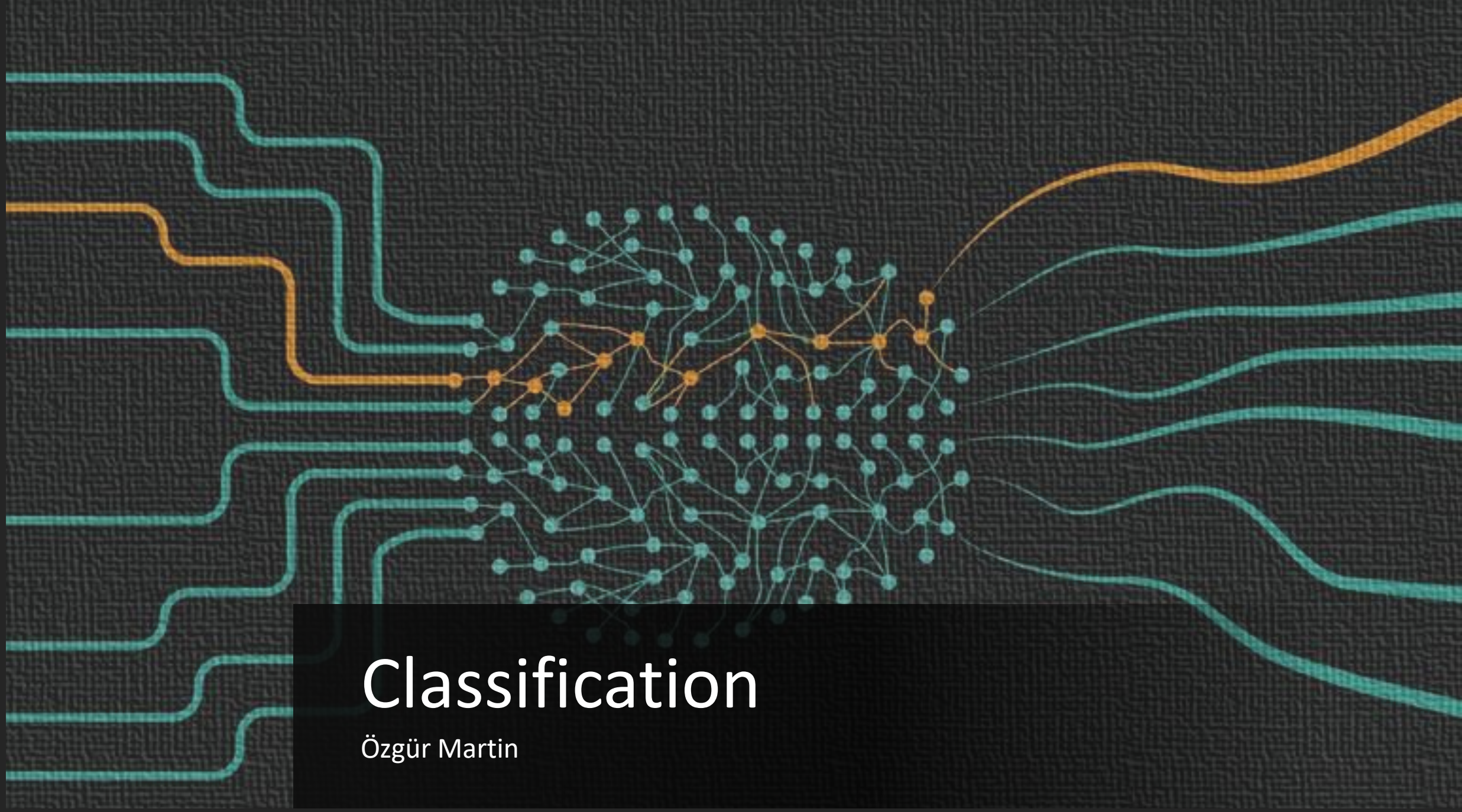
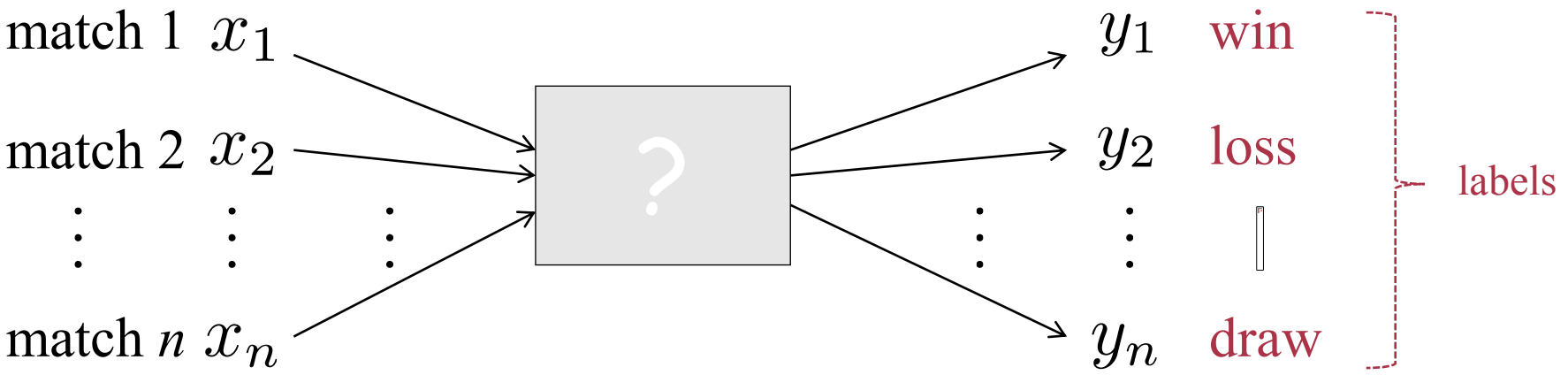


Classification

Özgür Martin

Classification



classification

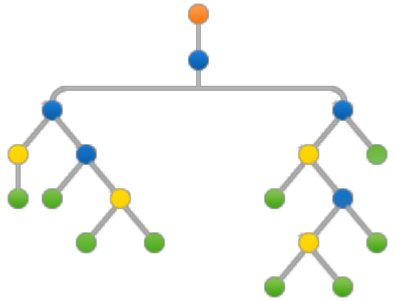
training data

$$\{(x_i, y_i) : 1, \ldots, n\}$$

Tree-Based Methods

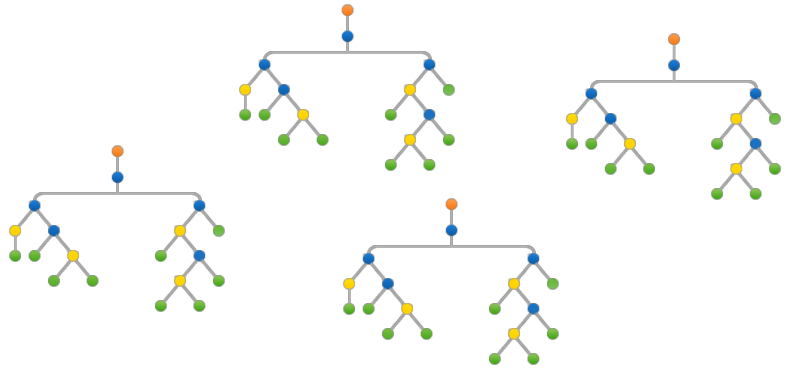
Objective: Divide the predictor space into a number of simple regions

Regression Trees
Classification Trees

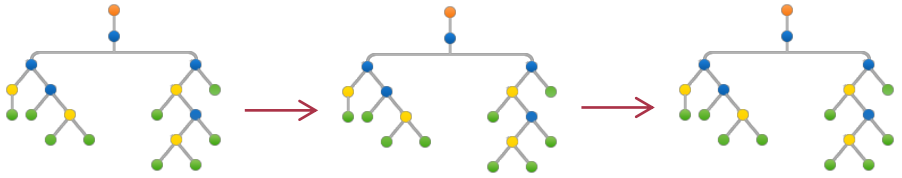


Ensemble Methods

Bagging & Random Forests

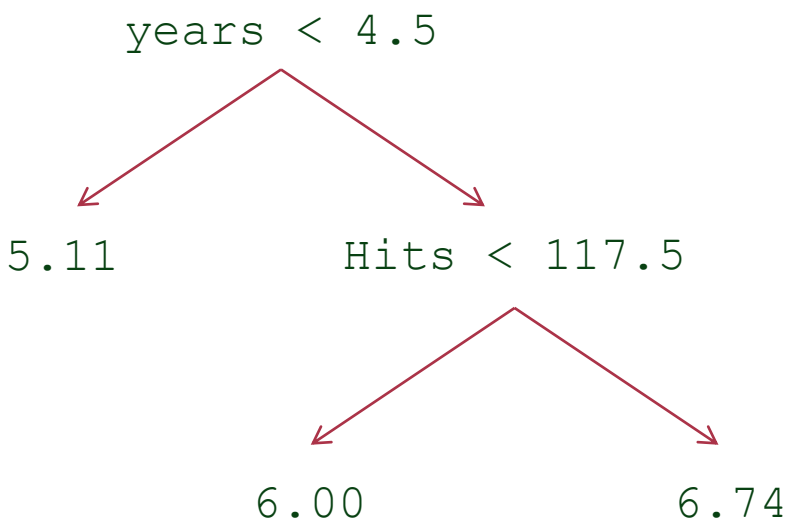


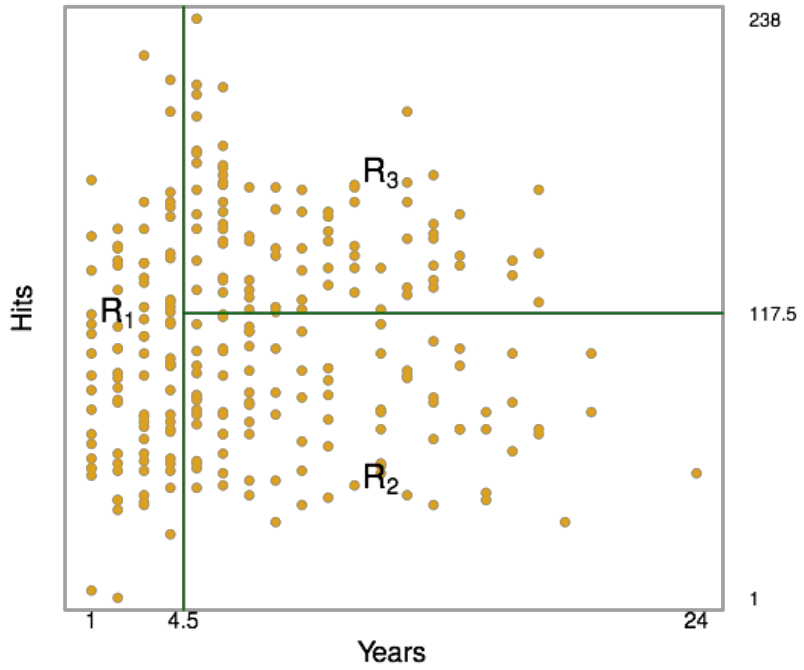
Boosting



Example Tree

Predicting the log salary of a player as a function of the number of hits and the years of experience





Regression Trees

$$X_1, X_2, \ldots, X_p \xrightarrow{\text{Distinct and nonoverlapping regions}} R_1, R_2, \ldots, R_J$$

Prediction: Mean of the response values for the training observations in R_j

$$R_1, R_2, \ldots, R_J \quad ?$$

Goal: Finding the regions such that RSS is minimized

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

$\hat{y}_{R_j}$: mean response within R_j

Regression Trees

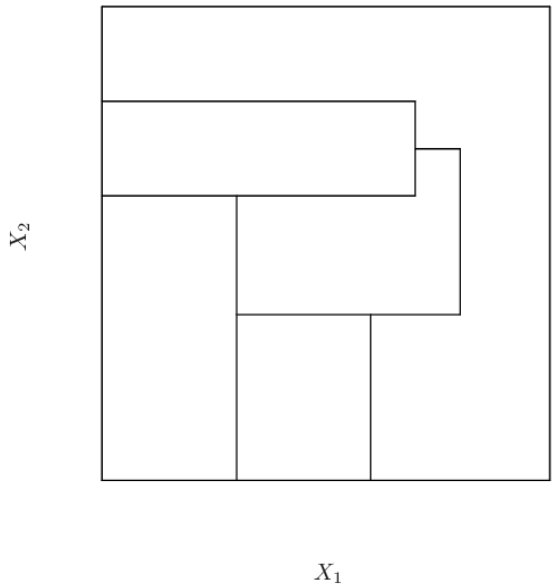
Recursive Binary Splitting

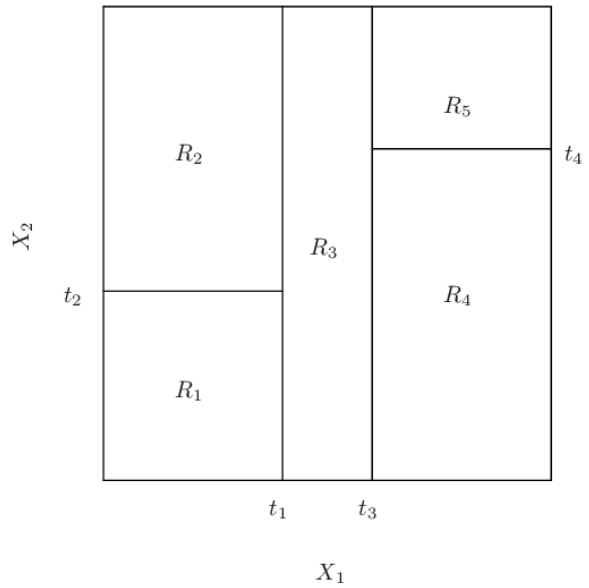
$$R_1(j,s) = \{X | X_j < s\} \qquad R_2(j,s) = \{X | X_j \geq s\}$$

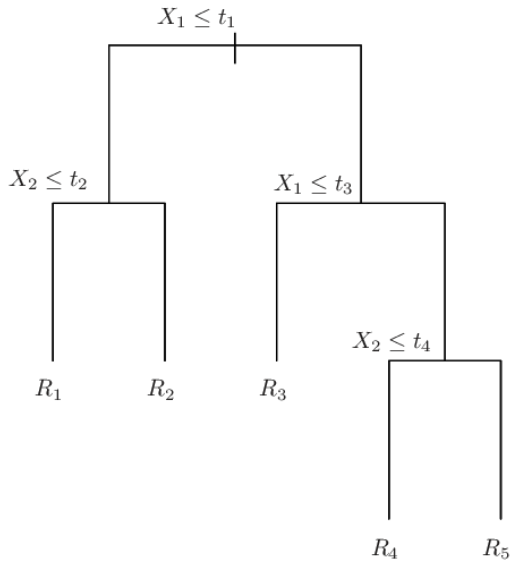
Find j and s that minimizes

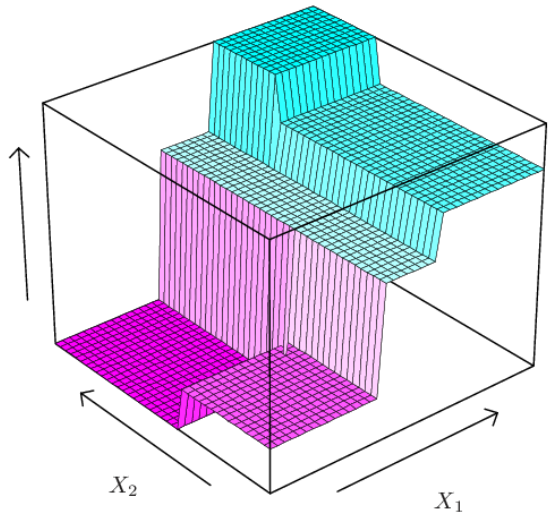
$$\sum_{i:x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$

Stop when each terminal node has a very small number of observations


X_2
X_1


R_1
R_2
R_3
R_4
R_5
t_1
t_2
t_3
t_4
X_2
X_1


$X_1 \leq t_1$
$X_2 \leq t_2$
$X_1 \leq t_3$
$X_2 \leq t_4$
R_1
R_2
R_3
R_4
R_5


X_2
X_1

Regression Trees

Tree Pruning

Goal: Avoiding overfitting with a fully grown or large tree
(Selecting a subtree that leads to a lowest test error rate)

$$\sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

$T \subset T_0$: subtree

$|T|$: number of terminal nodes in T

R_m : region corresponding to the mth terminal node

α : fixed parameter

$\alpha \uparrow$ $\quad$ $|T| \downarrow$

Use k-fold cross validation to choose α

Classification Trees

Similar to a regression tree but uses different error measures based on *purity* of a region

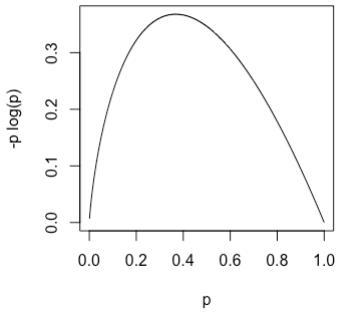
$\hat{p}_{mk}$: proportion of class-*k* training observations in the *m*th region

Classification Error Rate

$$E = 1 - \max_k \{\hat{p}_{mk}\}$$

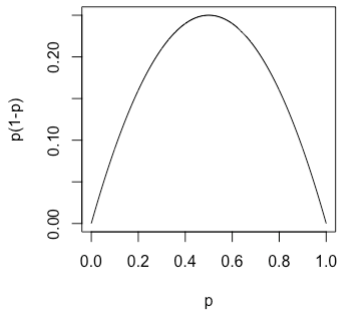
Entropy

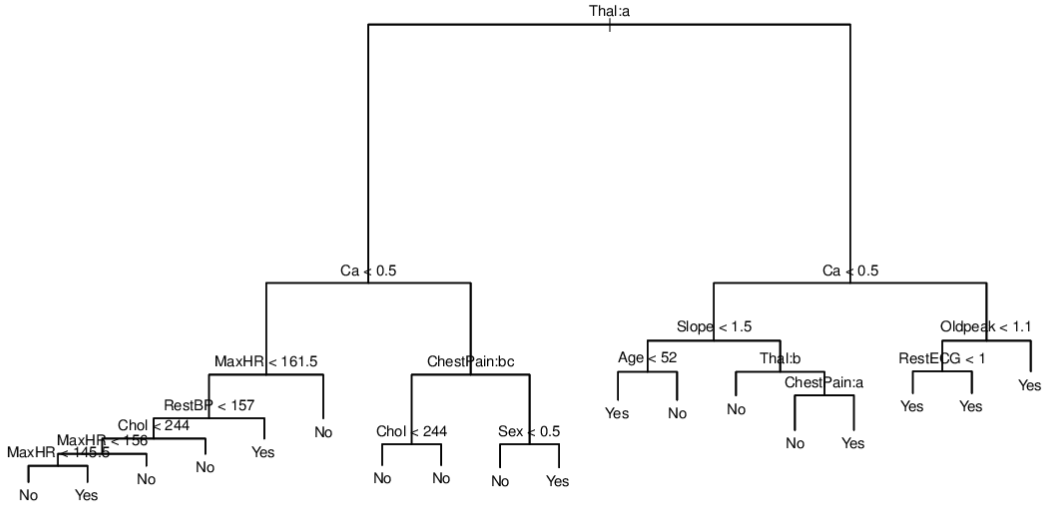
$$D = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$$

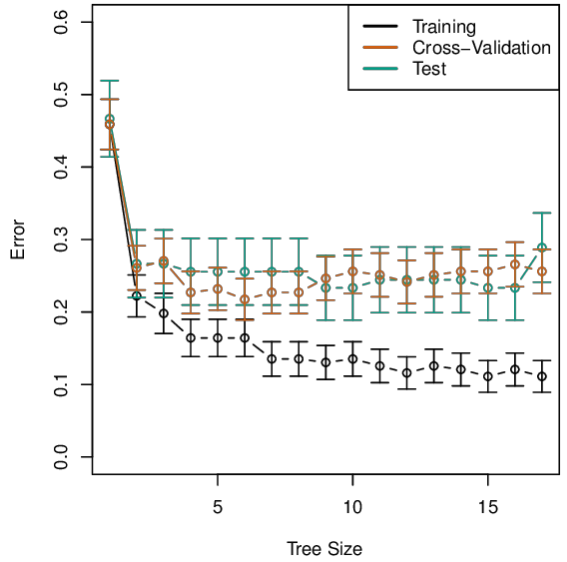


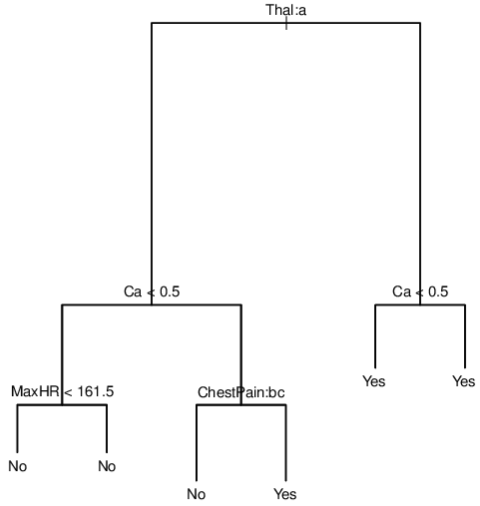
Gini Index

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$$




Thal:a
Ca < 0.5
Ca < 0.5
MaxHR < 161.5
ChestPain:bc
Slope < 1.5
Oldpeak < 1.1
RestBP < 157
Chol < 244
Age < 52
Thal:b
RestECG < 1
MaxHR < 156
MaxHR < 145.5
Chol < 244
Sex < 0.5
ChestPain:a
No
Yes

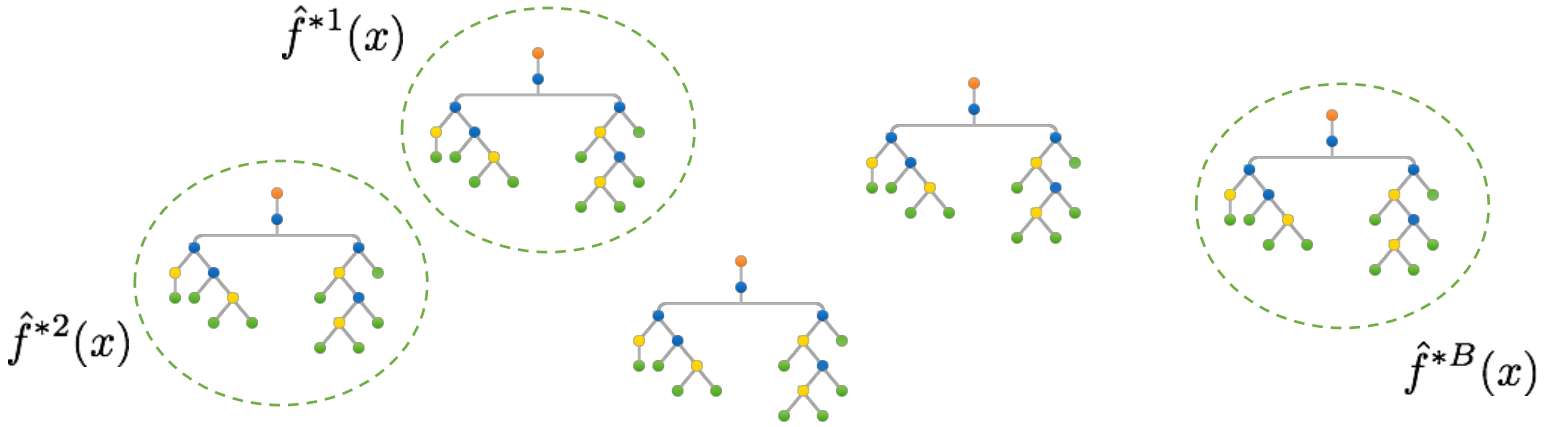

Training
Cross−Validation
Test
Error
0.0
0.1
0.2
0.3
0.4
0.5
0.6
5
10
15
Tree Size


Thal:a
Ca < 0.5
Ca < 0.5
MaxHR < 161.5
ChestPain:bc
No
Yes

Ensemble Methods

Bagging

Goal: Use bootstrapping to grow separate *deep* trees and average all the predictions (regression) or apply majority rule (classification) to reduce the variance



Regression

B : number of separate training sets

$\hat{f}^{*b}(x)$: prediction obtained with the bth training set

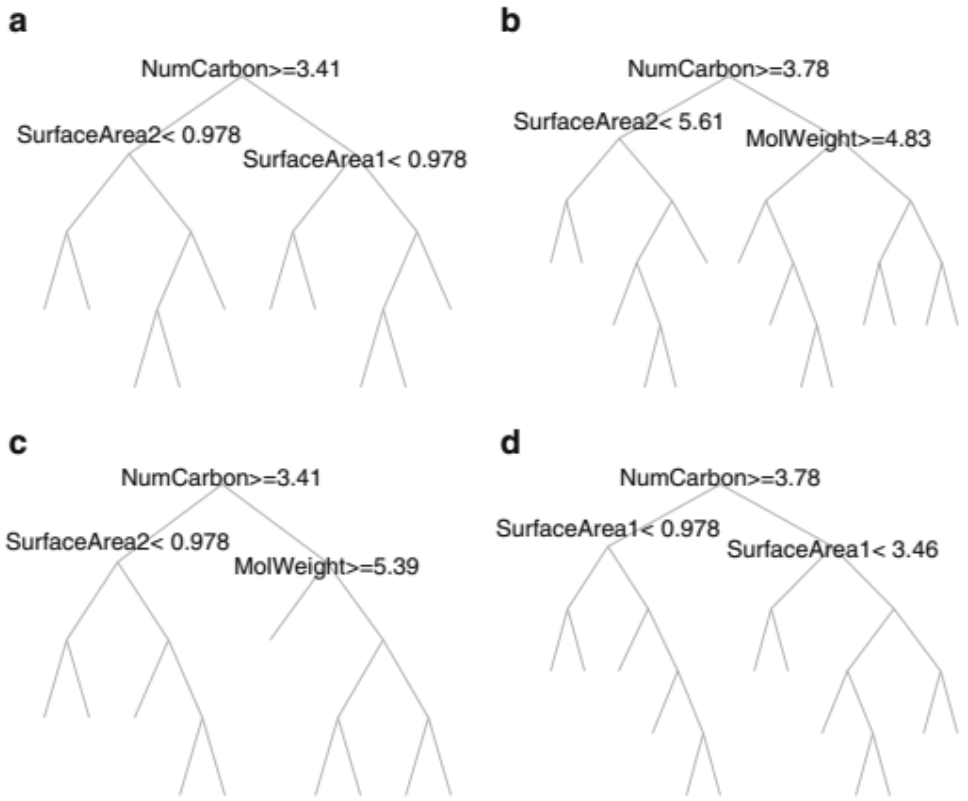
$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{*b}(x)$$

Ensemble Methods

Random Forests

Goal: Smart bagging to *decorrelate* the trees – only a random sample of predictors is chosen as candidates for splitting

Why?*

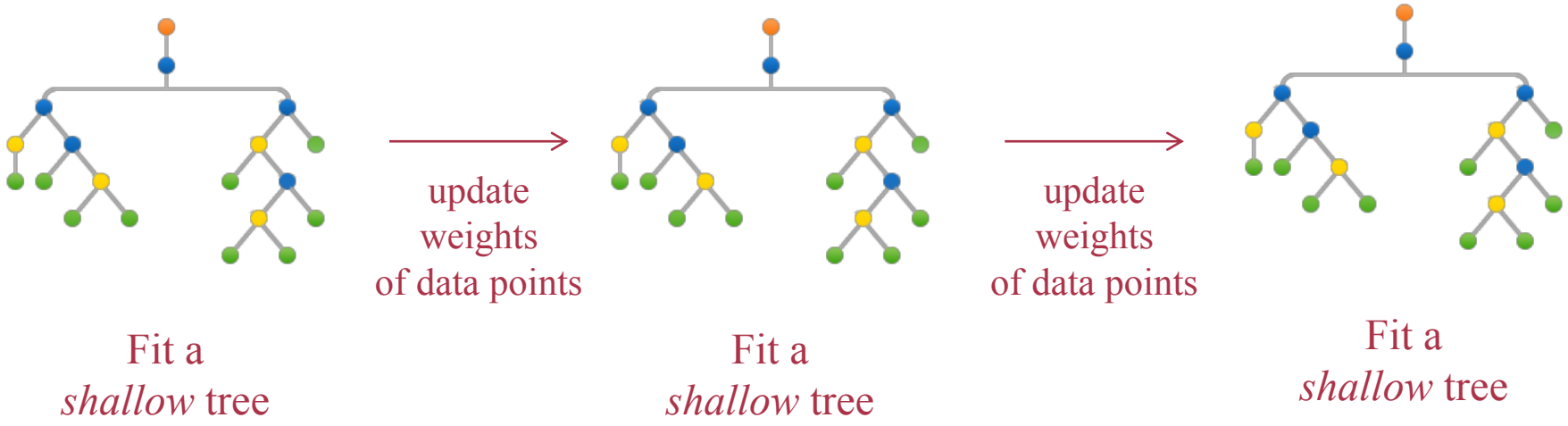


**Applied Predictive Modeling*, M. Kuhn, K. Johnson., Springer, 2013, pg. 195.

Ensemble Methods

Boosting

Idea: Combining mediocre classifiers sequentially to boost their collective performance through weighted data sampling



Update Rule: Give more weights to incorrectly classified points

Ensemble Methods

AdaBoost – Binary Classification {+1, -1}

Each sample has the same starting weight (*1/n*)

for *k=1 to K* **do**

Fit a tree with *d* splits using the weighted samples and compute misclassification error (ϵ_k)

Compute stage weight value $\ln \frac{1-\epsilon_k}{\epsilon_k}$

Update the sample weights – give more weights to incorrectly classified samples

end

Compute the boosted classifier's prediction for each sample by multiplying the *k*th stage value by the *k*th model prediction and add these quantities across k. If the sum is positive, then classify the sample as +1 otherwise as -1

Algorithm 8.2 in the textbook gives a boosting example for **regression trees**

Ensemble Methods

AdaBoost – Binary Classification {+1, -1}

Algorithm 1: AdaBoost

1 Initialize weights $w_i^1 = \frac{1}{n}$ for $i = 1, \ldots, n$

2 **for** $k = 1, \ldots, K$ **do**

3 Fit a classifier $y_k(x)$ that minimizes the weighted classification error

$$\sum_{i=1}^{n} w_i^k I(y_k(x_i) \neq y_i)$$

4 Evaluate the relative weight of the misclassified data points

$$\varepsilon^k = \frac{\sum_{i=1}^{n} w_i^k I(y_k(x_i) \neq y_i)}{\sum_{i=1}^{n} w_i^k}$$

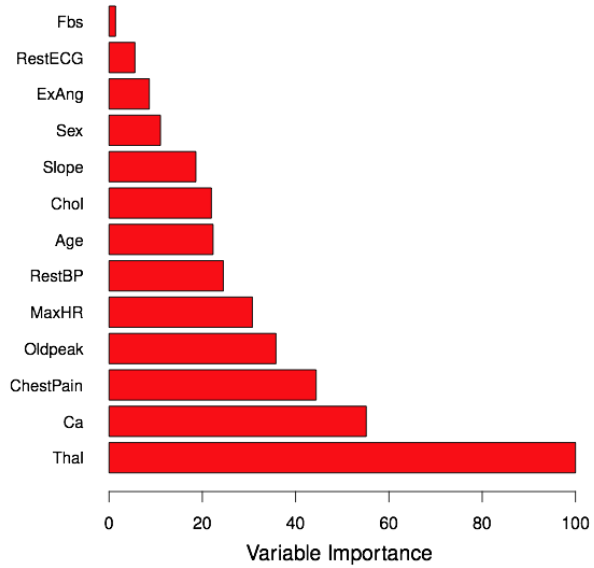
5 Set $\alpha_k = \ln \frac{1-\varepsilon_k}{\varepsilon_k}$

6 Update $w_i^{k+1} = w_i^k e^{\alpha_k I(y_k(x_i) \neq y_i)}$ for $i = 1, \ldots, n$

7 Output sign $\left(\sum_{k=1}^{K} \alpha_k y_k(x) \right)$

Ensemble Methods

- Both boosting and bagging can be applied to different learning methods
- While bagging allows direct parallel implementation, the sequential structure of boosting prevents parallelization
- Ensemble methods cause loss of interpretability
- Variable importance plots can be used



Variable importance is computed by sorting the predictors according to the mean decrease they achieve in Gini index or entropy (normalized)

Tree-Based Methods in Action

```
[9]: # Required packages
     from sklearn.tree import DecisionTreeClassifier
     from sklearn.datasets import make_classification
     from sklearn.ensemble import BaggingClassifier
     from sklearn.ensemble import RandomForestClassifier
     from sklearn.ensemble import AdaBoostClassifier
```

Adding packages and functions

```
[10]: # Generate and extract features and target data
      X, y = make_classification(n_samples=2000,
                                 n_features=4,
                                 n_informative=2,
                                 n_redundant=0,
                                 random_state=42,
                                 shuffle=False)

      X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.7, shuffle=False
      )
```

Generating and splitting the dataset

```
[11]: # Create decision tree and train it
      DTClassifier = DecisionTreeClassifier(criterion = 'gini',
                                            splitter = 'best',
                                            max_depth = None,
                                            min_samples_leaf = 100)
      DTClassifier.fit(X_train, y_train)
      DTClassifier.score(X_test, y_test)

[11]: 0.8521428571428571
```

Predicting with Decision Tree

Tree-Based Methods in Action

```
[12]: # Bagging
      bagging = BaggingClassifier(DTClassifier,
                                  n_estimators = 20,
                                  max_samples = 1.0,
                                  bootstrap = True,
                                  bootstrap_features = False)
      bagging.fit(X_train, y_train)
      bagging.score(X_test, y_test)

[12]: 0.8892857142857142
```

Predicting with bagging

```
[13]: # Random Forest
      RF = RandomForestClassifier(criterion = 'gini',
                                  n_estimators=10,
                                  max_depth=None,
                                  min_samples_leaf = 100,
                                  random_state=0)
      RF.fit(X_train, y_train)
      RF.score(X_test, y_test)

[13]: 0.8607142857142858
```

Predicting with Random Forest

```
[14]: # Adaboost
      ABC = AdaBoostClassifier(n_estimators=100, random_state=0)
      ABC.fit(X_train, y_train)
      ABC.score(X_test, y_test)

[14]: 0.7407142857142858
```

Predicting with Adaboost

Support Vector Machines

$$(x_i, y_i), i = 1, 2, \ldots, n$$

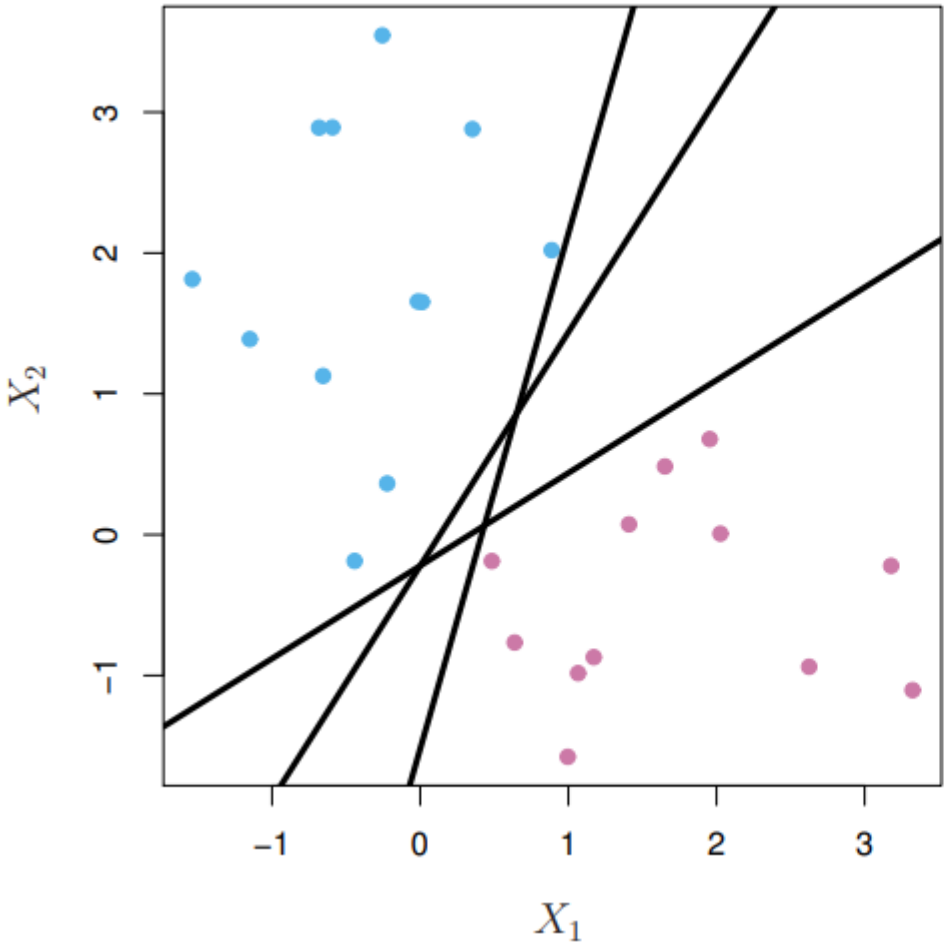
$$x_i \in \mathbb{R}^p \qquad\qquad y_i \in \{-1, +1\}$$

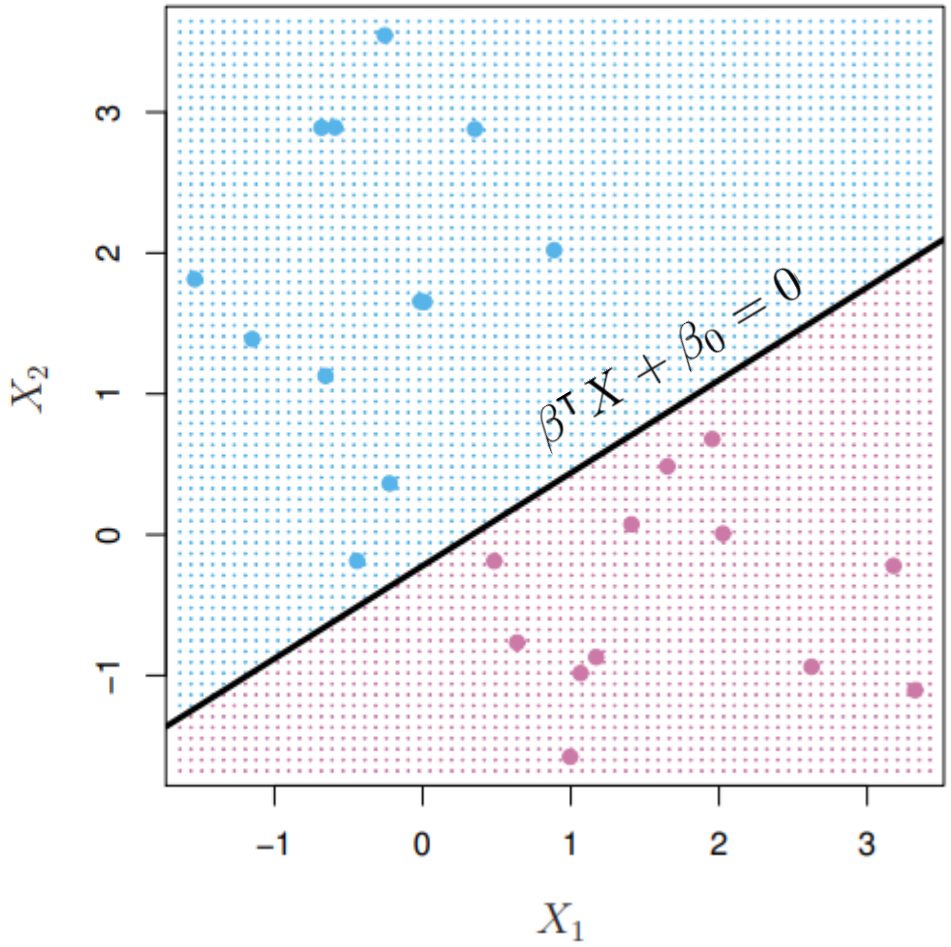
Idea: Come up with a hyperplane so that the data points are classified according to the side of the hyperplane (halfspace) that they reside

Hyperplane

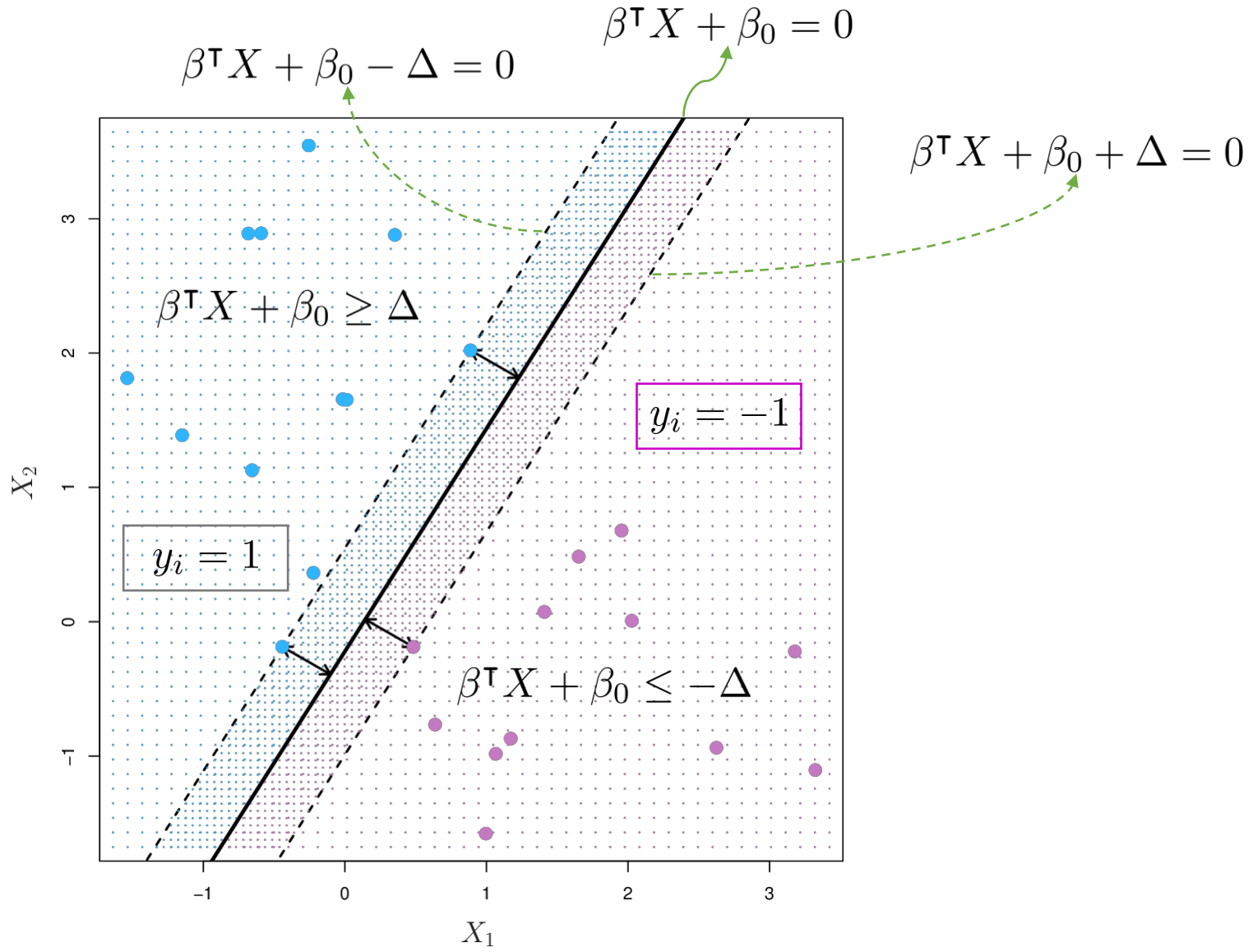
$$\{X \in \mathbb{R}^p : \beta^\intercal X + \beta_0 = 0 \text{ for some } \beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}\}$$

Perfectly Separable Data





Perfectly Separable Data



Perfectly Separable Data

Objective: Choose β and Δ such that the distance between the two hyperplanes

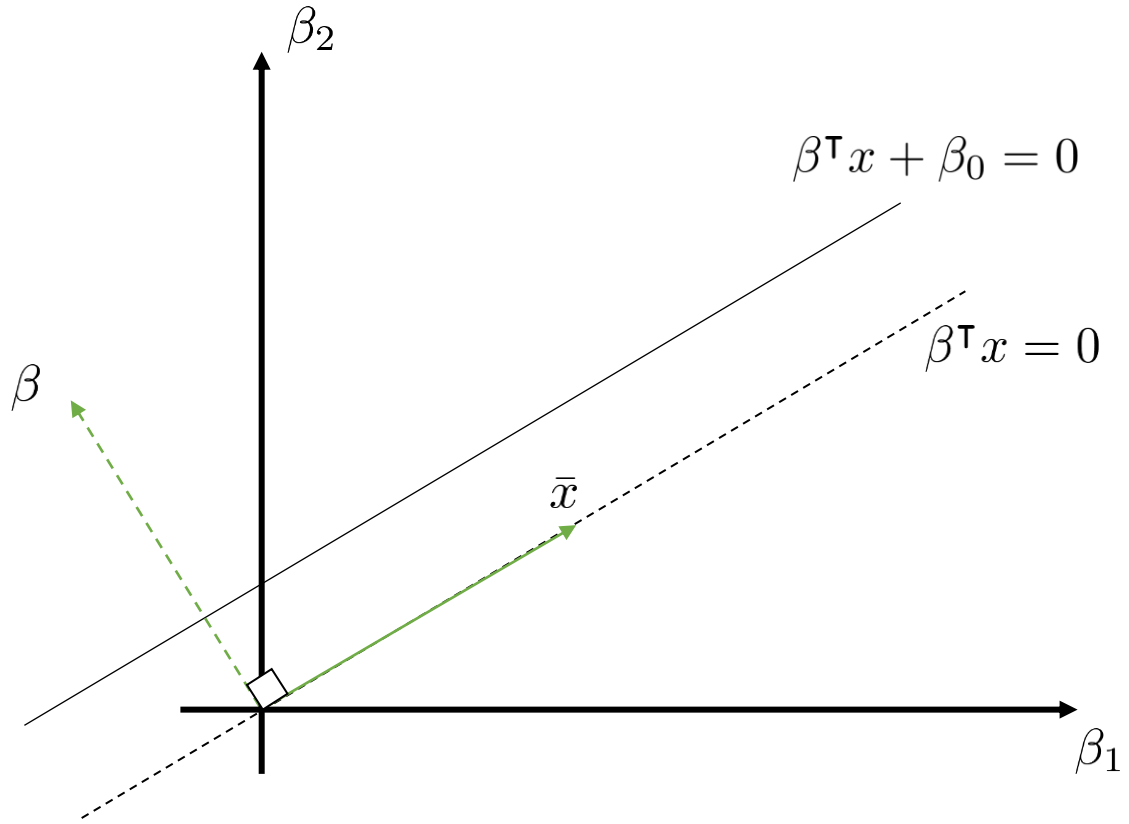
$$\beta^\intercal X + \beta_0 - \Delta = 0 \quad \text{and} \quad \beta^\intercal X + \beta_0 + \Delta = 0$$

is as large as possible (with scaling we can set $\Delta = 1$)

$$(x_i, y_i) \begin{cases} y_i = 1 \longrightarrow \beta^\intercal x_i + \beta_0 - 1 \geq 0 \\ y_i = -1 \longrightarrow \beta^\intercal x_i + \beta_0 + 1 \leq 0 \end{cases}$$

$$y_i(\beta^\intercal x_i + \beta_0) \geq 1$$

Perfectly Separable Data



β is normal to the subspace

$$\beta^\intercal x = 0$$

β is also normal to

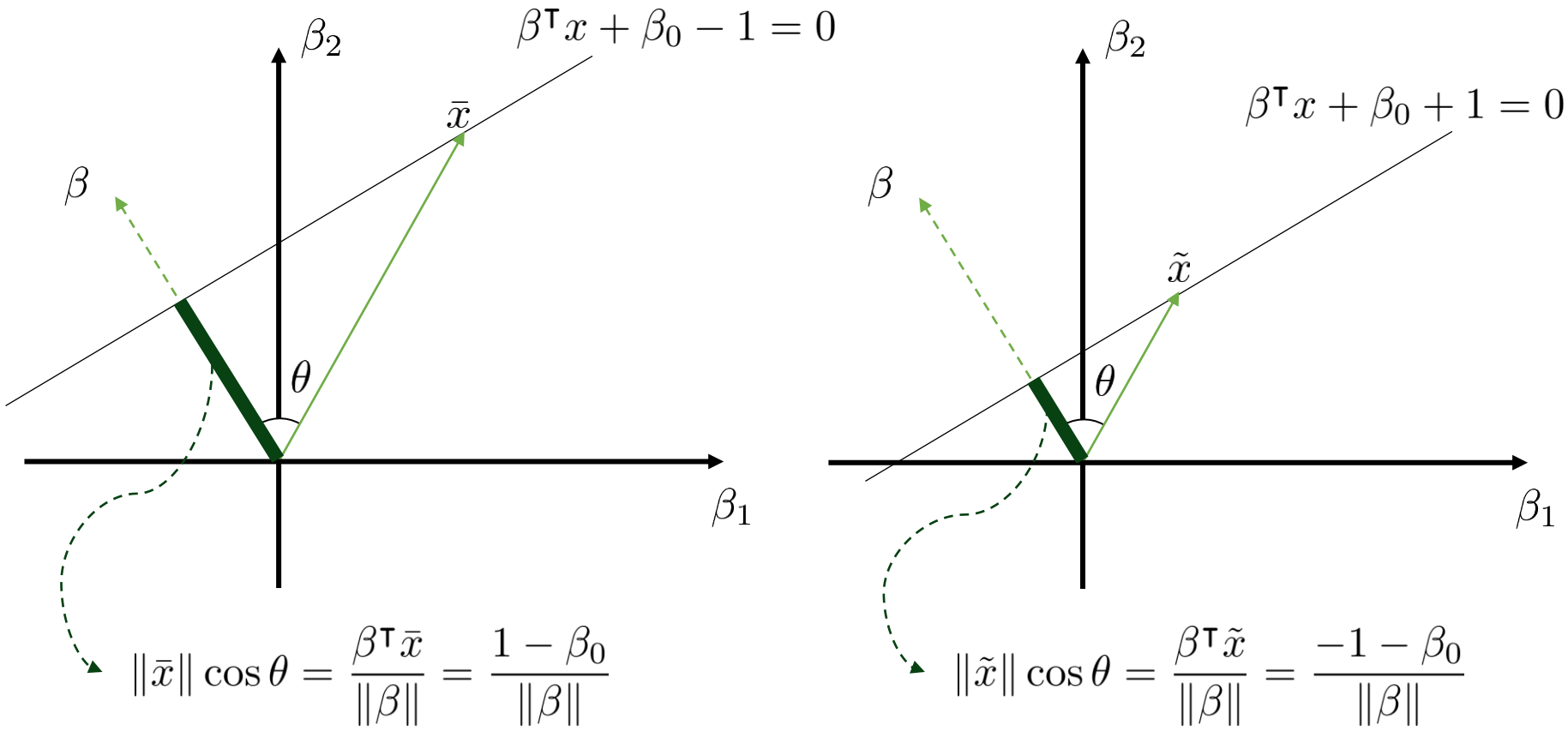
$$\beta^\intercal x + \beta_0 = 1$$

and

$$\beta^\intercal x + \beta_0 = -1$$

$$\beta = (\beta_1, \beta_2)^\intercal$$

Perfectly Separable Data



The distance between the two hyperplanes: $\frac{1 - \beta_0}{\|\beta\|} - \frac{-1 - \beta_0}{\|\beta\|} = \frac{2}{\|\beta\|}$

Perfectly Separable Data

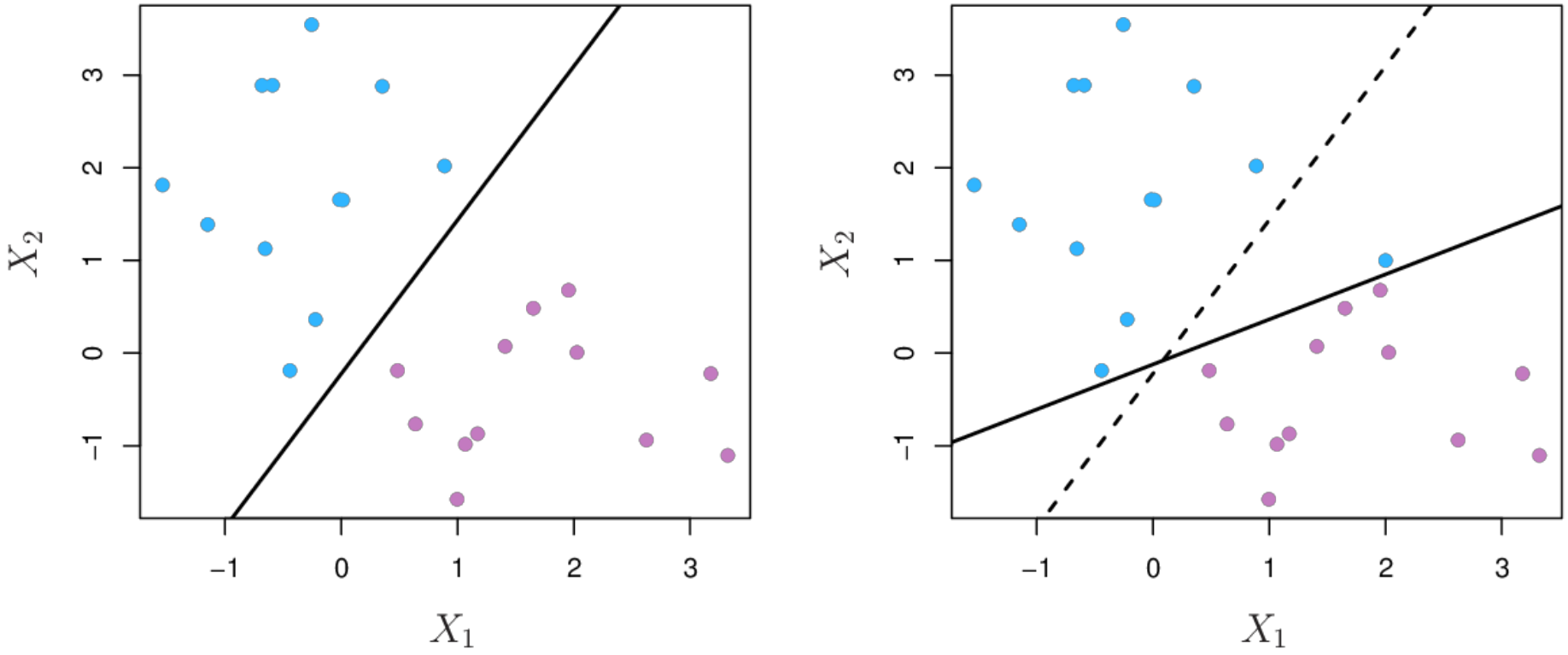
$$
\begin{aligned}
&\text{maximize} && \frac{2}{\|\beta\|} \\
&\text{subject to} && y_i(\beta^\intercal x_i + \beta_0) \geq 1, \quad i = 1, 2, \ldots, n
\end{aligned}
$$

$$
\equiv
\begin{aligned}
&\text{minimize} && \tfrac{1}{2}\|\beta\| \\
&\text{subject to} && y_i(\beta^\intercal x_i + \beta_0) \geq 1, \quad i = 1, 2, \ldots, n
\end{aligned}
$$

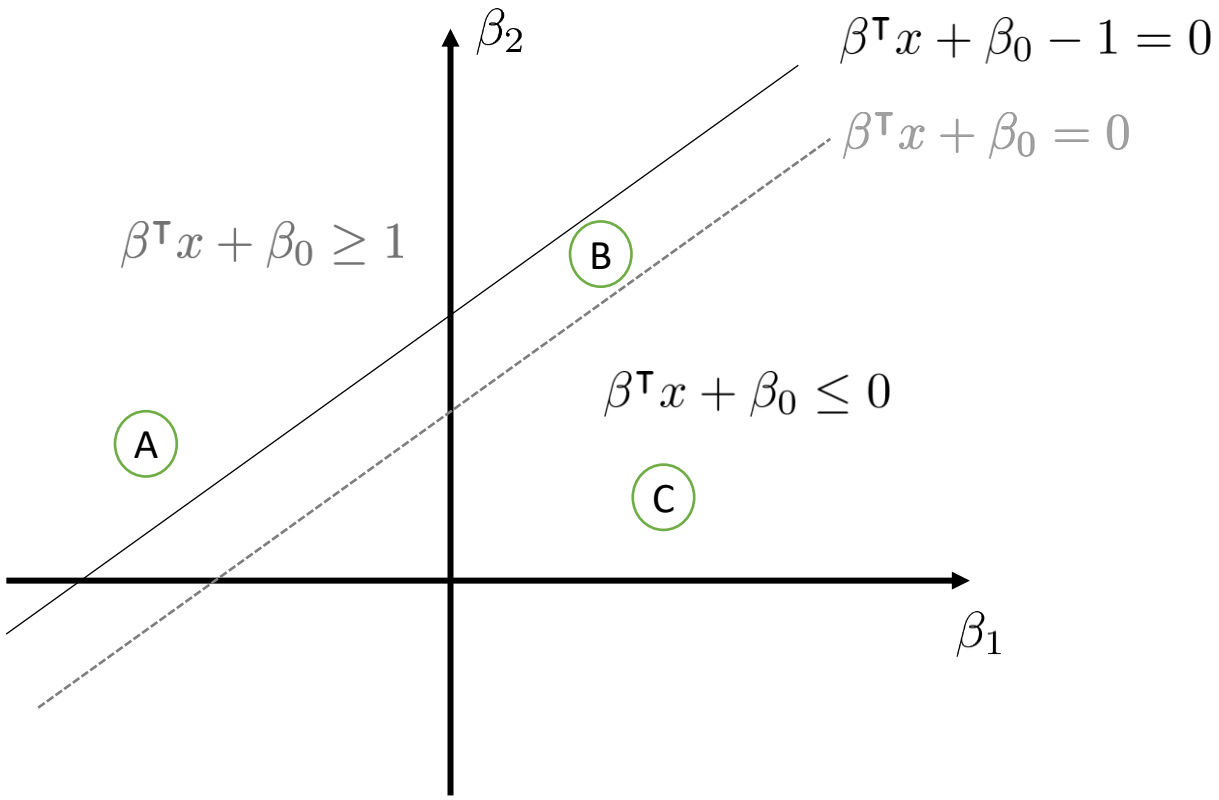
$$
\equiv
\begin{aligned}
&\text{minimize} && \tfrac{1}{2}\beta^\intercal\beta \\
&\text{subject to} && y_i(\beta^\intercal x_i + \beta_0) \geq 1, \quad i = 1, 2, \ldots, n
\end{aligned}
$$

This is an optimization problem with convex quadratic objective function and linear constraints. This type of problem is known as *quadratic programming*.

Perfectly Separable Data



Not Perfectly Separable Data



(x_i, y_i)

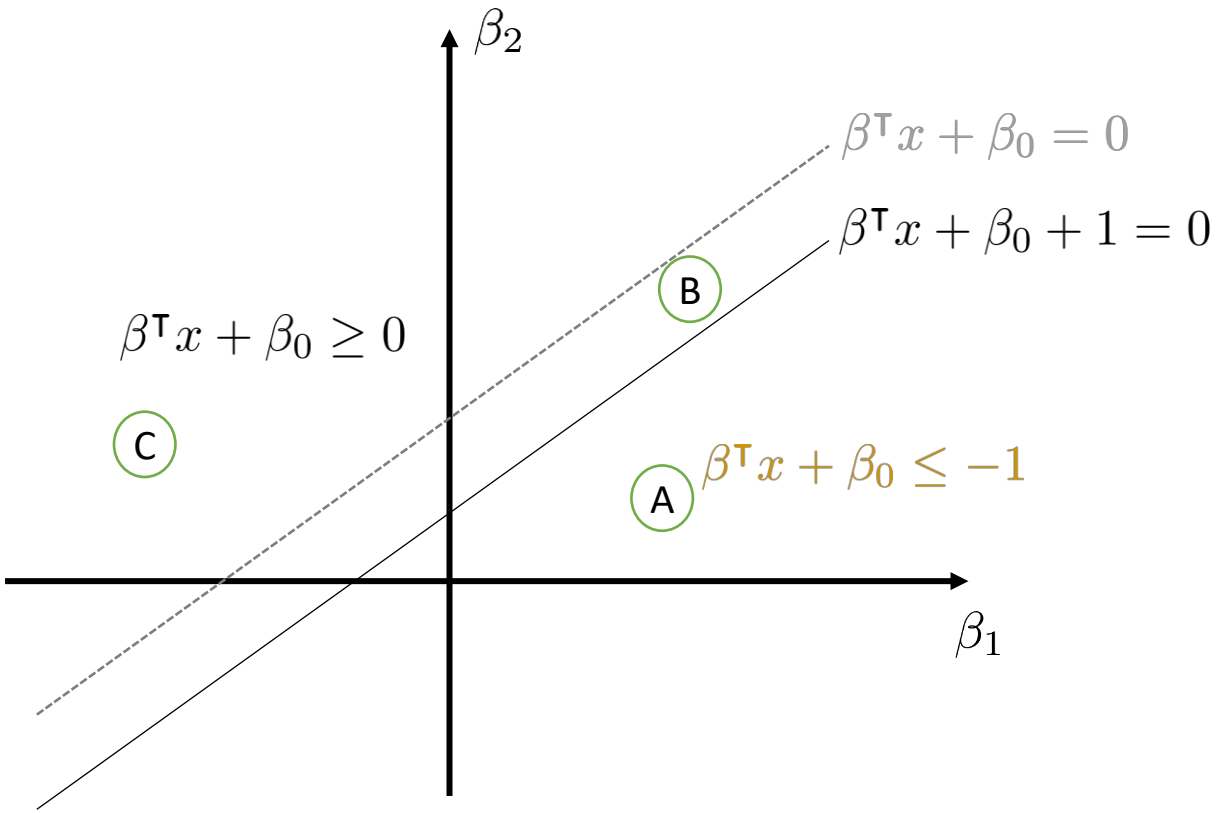
$y_i = 1$

A $\quad y_i(\beta^T x_i + \beta_0) \geq 1 - \varepsilon, \quad \varepsilon = 0$

B $\quad y_i(\beta^T x_i + \beta_0) \geq 1 - \varepsilon, \quad 0 < \varepsilon < 1$

C $\quad y_i(\beta^T x_i + \beta_0) \geq 1 - \varepsilon, \quad \varepsilon \geq 1$

Not Perfectly Separable Data



(x_i, y_i)

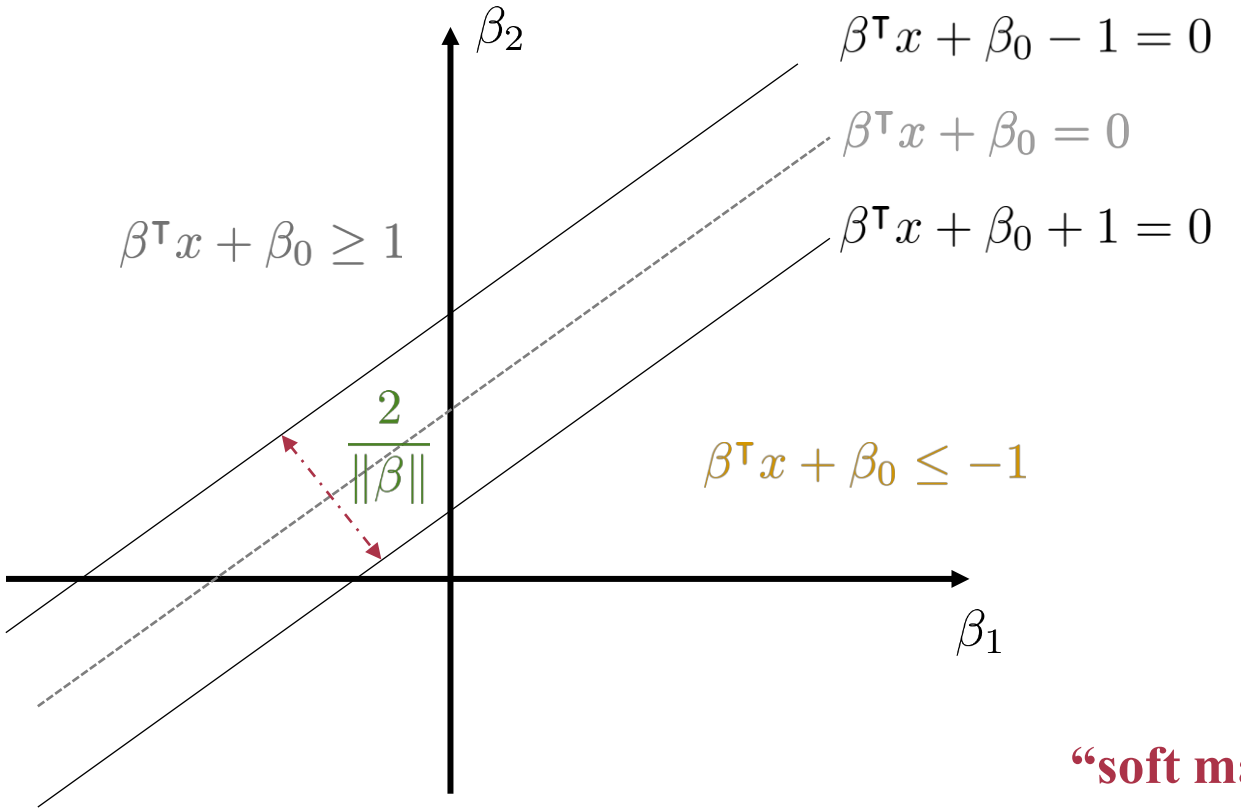
$y_i = -1$

A $y_i(\beta^T x_i + \beta_0) \geq 1 - \varepsilon, \quad \varepsilon = 0$

B $y_i(\beta^T x_i + \beta_0) \geq 1 - \varepsilon, \quad 0 < \varepsilon < 1$

C $y_i(\beta^T x_i + \beta_0) \geq 1 - \varepsilon, \quad \varepsilon \geq 1$

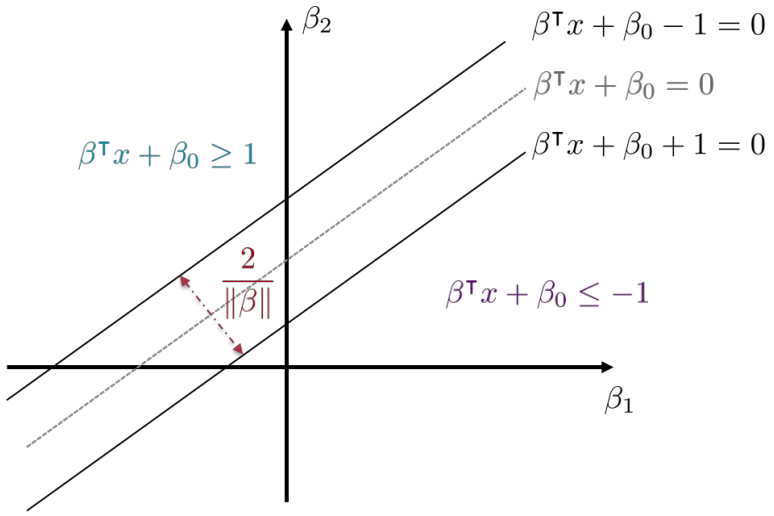
Not Perfectly Separable Data



"soft margin"

$$y_i(\beta^\intercal x_i + \beta_0) \geq 1 - \varepsilon, \quad \varepsilon = 0$$
$$y_i(\beta^\intercal x_i + \beta_0) \geq 1 - \varepsilon, \quad 0 < \varepsilon < 1$$
$$y_i(\beta^\intercal x_i + \beta_0) \geq 1 - \varepsilon, \quad \varepsilon \geq 1$$

Not Perfectly Separable Data

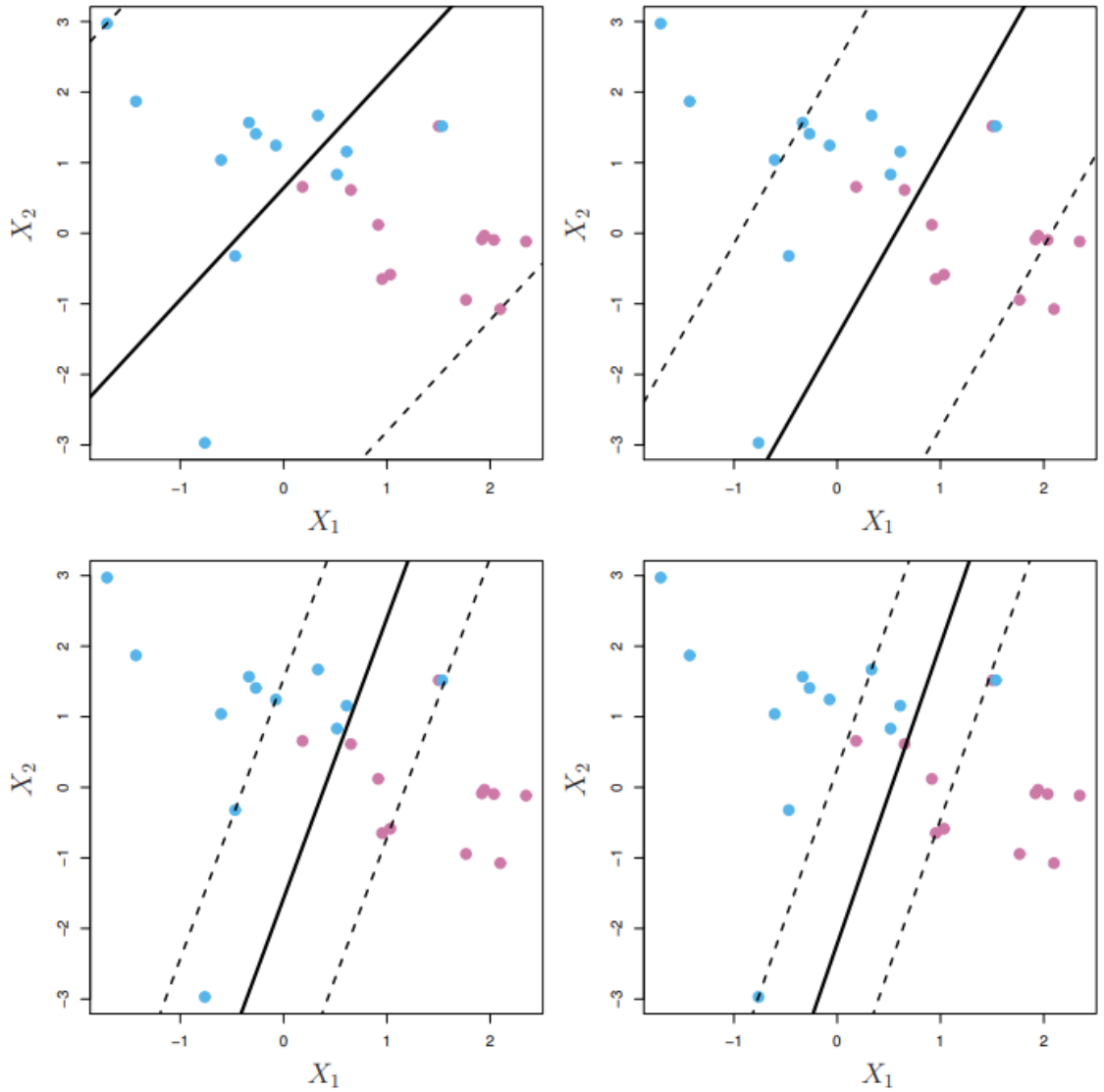


$$y_i(\beta^\intercal x_i + \beta_0) \geq 1 - \varepsilon, \quad \varepsilon = 0$$
$$y_i(\beta^\intercal x_i + \beta_0) \geq 1 - \varepsilon, \quad 0 < \varepsilon < 1$$
$$y_i(\beta^\intercal x_i + \beta_0) \geq 1 - \varepsilon, \quad \varepsilon \geq 1$$

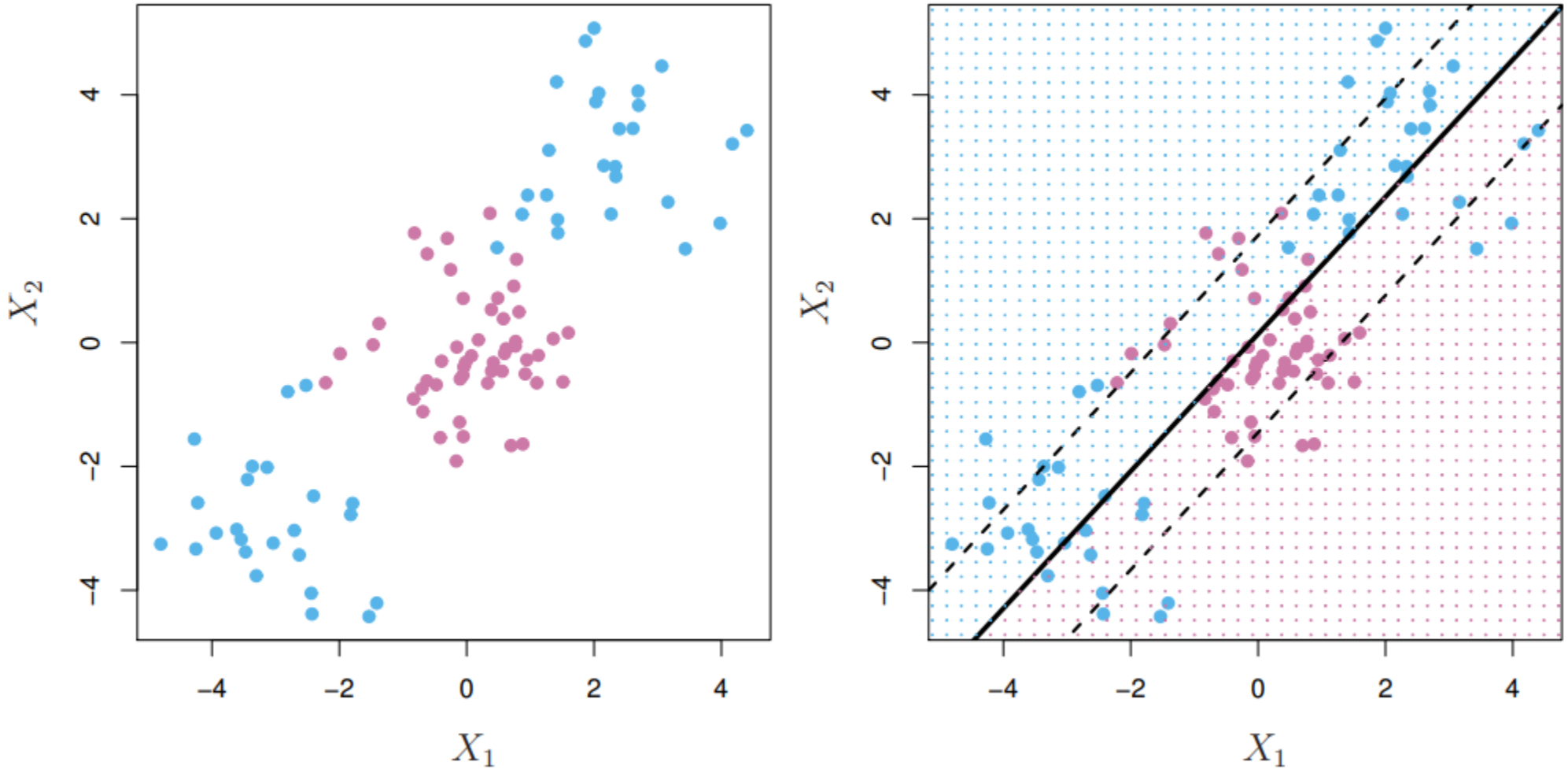
$$\begin{aligned} \text{minimize} \quad & \tfrac{1}{2}\beta^\intercal\beta + c\sum_{i=1}^{n}\varepsilon_i^d \\ \text{subject to} \quad & y_i(\beta^\intercal x_i + \beta_0) \geq 1 - \varepsilon_i, \quad i = 1, \ldots, n \\ & \varepsilon_i \geq 0, \quad i = 1, \ldots, n \end{aligned}$$

Here c is a tuning parameter that we set by cross-validation.
($d = 1$ and $d = 2$ are quite common in practice)

Not Perfectly Separable Data



Path to Kernels



Path to Kernels

$$\begin{aligned} &\text{minimize} && \tfrac{1}{2}\beta^\intercal\beta \\ &\text{subject to} && y_i(\beta^\intercal x_i + \beta_0) \geq 1, \quad i = 1, 2, \ldots, n \end{aligned}$$

Lagrangian Function

Lagrange multipliers

$$\mathcal{L}(\beta, \beta_0; \alpha) = \frac{1}{2}\beta^\intercal\beta - \sum_{i \in \mathcal{A}} \alpha_i \left(y_i(\beta^\intercal x_i + \beta_0) - 1\right)$$

set of active constraints

Optimality Conditions

$$\nabla_\beta \mathcal{L}(\beta, \beta_0; \alpha) = \beta - \sum_{i \in \mathcal{A}} \alpha_i y_i x_i = 0 \implies \beta = \sum_{i \in \mathcal{A}} \alpha_i y_i x_i$$

$$\frac{\partial \mathcal{L}(\beta, \beta_0; \alpha)}{\partial \beta_0} = -\sum_{i \in \mathcal{A}} \alpha_i y_i = 0 \implies \sum_{i \in \mathcal{A}} \alpha_i y_i = 0$$

$$\begin{aligned} \mathcal{L}(\beta, \beta_0; \alpha) = \quad & \tfrac{1}{2}\left(\textstyle\sum_{i \in \mathcal{A}} \alpha_i y_i x_i\right)^\intercal \left(\textstyle\sum_{j \in \mathcal{A}} \alpha_j y_j x_j\right) - \left(\textstyle\sum_{i \in \mathcal{A}} \alpha_i y_i \left(\textstyle\sum_{j \in \mathcal{A}} \alpha_j y_j x_j\right)^\intercal x_i\right) \\ & - \textstyle\sum_{i \in \mathcal{A}} \alpha_i y_i \beta_0 + \textstyle\sum_{i \in \mathcal{A}} \alpha_i \\ = \quad & \textstyle\sum_{i \in \mathcal{A}} \alpha_i - \tfrac{1}{2} \textstyle\sum_{i \in \mathcal{A}} \textstyle\sum_{j \in \mathcal{A}} \alpha_i \alpha_j y_i y_j \boxed{x_i^\intercal x_j} \quad ! \end{aligned}$$

Kernels

$$\mathcal{L}(\beta, \beta_0; \alpha) = \sum_{i \in \mathcal{A}} \alpha_i - \frac{1}{2} \sum_{i \in \mathcal{A}} \sum_{j \in \mathcal{A}} \alpha_i \alpha_j y_i y_j \boxed{x_i^\intercal x_j}^*$$

All we need is to calculate these inner products!

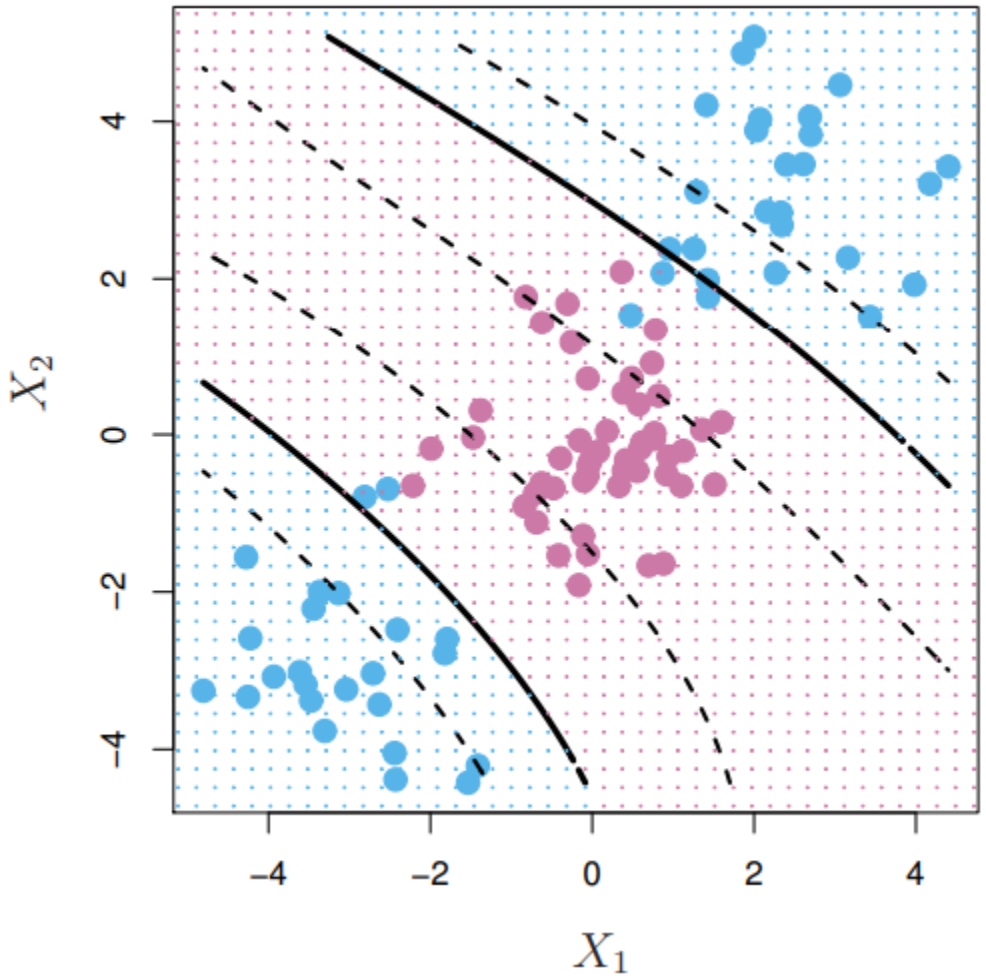
Idea: Replace inner products with kernels that measure the similarity of two observations in higher dimensions.

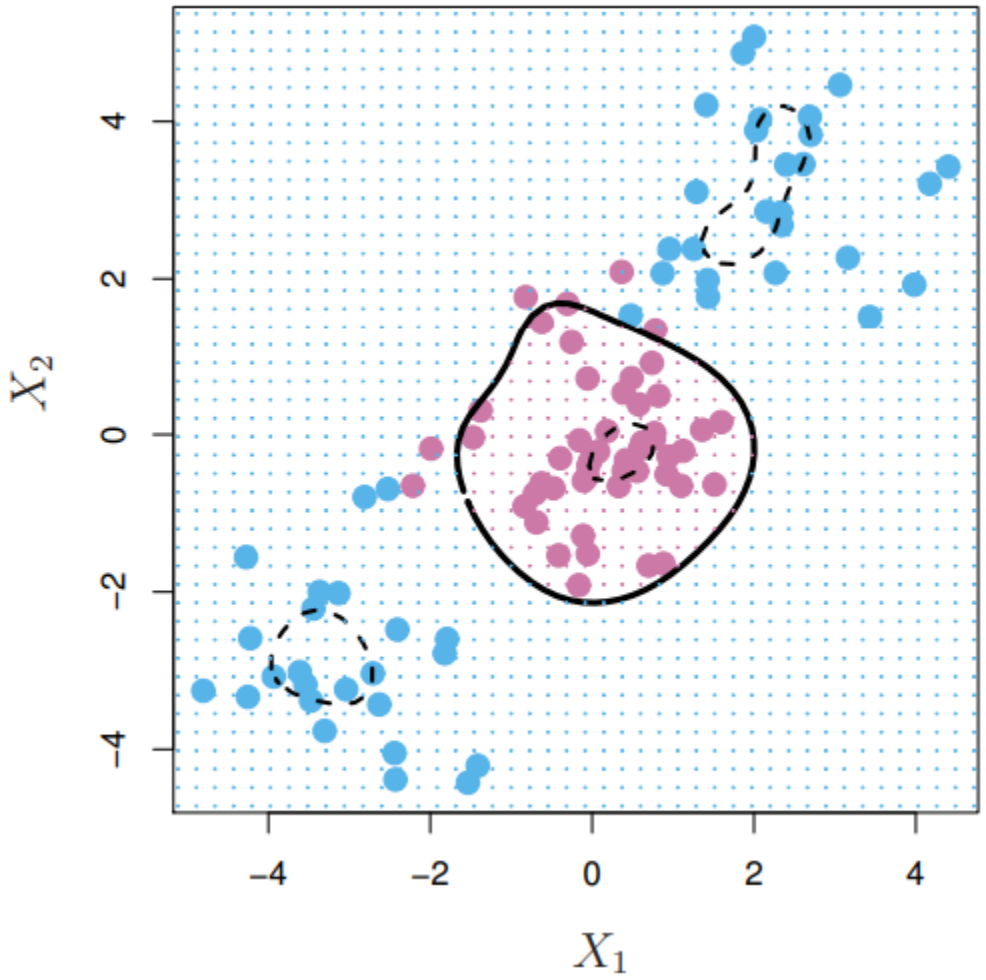
Linear Kernel* $\quad K(x_i, x_j) = x_i^\intercal x_j$

Polynomial Kernel $\quad K(x_i, x_j) = (1 + x_i^\intercal x_j)^d$

Radial Kernel $\quad K(x_i, x_j) = e^{-\gamma \|x_i - x_j\|^2}, \ \gamma > 0$

Kernels

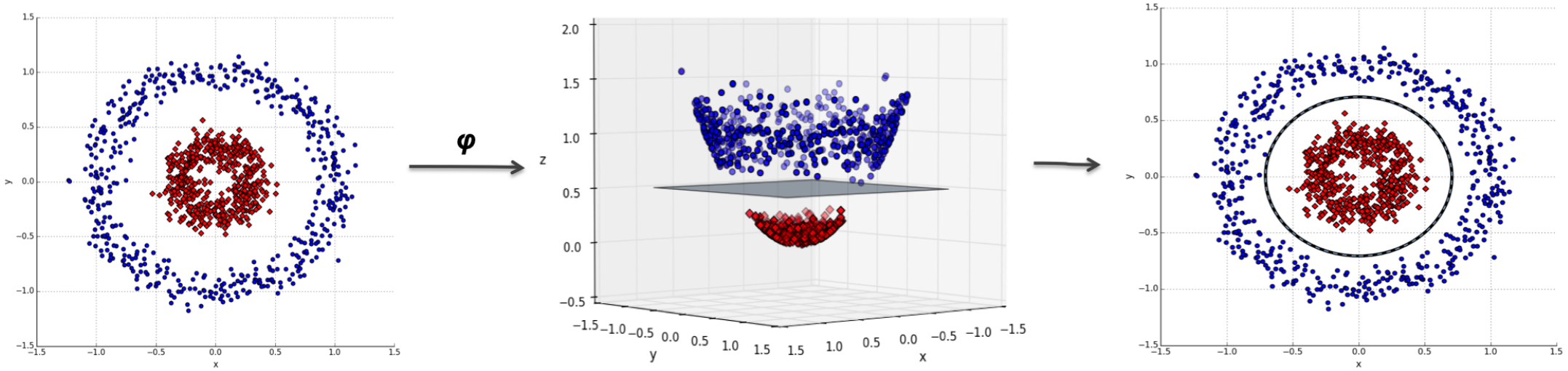




Kernels

$$\varphi : \mathbb{R}^p \to \mathbb{R}^q$$

$$K(x_i, x_j) = \varphi(x_i)^\intercal \varphi(x_j)$$



Kernels

Polynomial Kernel $K(x_i, x_j) = (1 + x_i^\intercal x_j)^d$

$x_i, x_j \in \mathbb{R}^2$

$d = 2$

$$K(x_i, x_j) = (1 + x_i^\intercal x_j)^2 = \quad 1 + x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2} + 2x_{i1}x_{i2}x_{j1}x_{j2}$$

$\varphi : \mathbb{R}^2 \to \mathbb{R}^6$

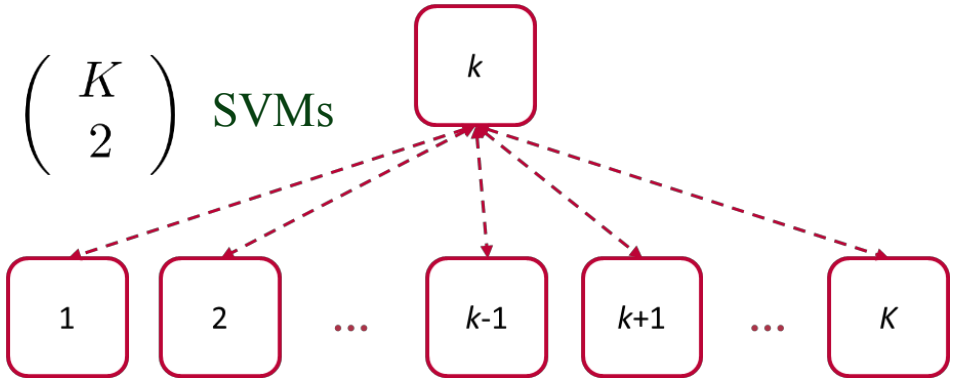
$$\varphi(x_i) = (1, x_{i1}^2, x_{i2}^2, \sqrt{2}x_{i1}, \sqrt{2}x_{i2}, \sqrt{2}x_{i1}x_{i2})^\intercal$$

$$K(x_i, x_j) = \varphi(x_i)^\intercal \varphi(x_j) = (1 + x_i^\intercal x_j)^2$$

Multiclass Classification

One-Versus-One (All-Pairs)

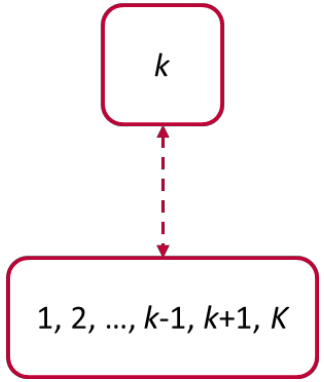
$\binom{K}{2}$ SVMs



x_0 is assigned to the most frequently assigned class

One-Versus-Rest

K SVMs



$$\beta_0^{(k)} + \beta^{(k)\mathsf{T}} x_0$$

$$\begin{matrix} \beta_0^{(1)} + \beta^{(1)\mathsf{T}} x_0 \\ \beta_0^{(2)} + \beta^{(2)\mathsf{T}} x_0 \\ \vdots \\ \beta_0^{(K)} + \beta^{(K)\mathsf{T}} x_0 \end{matrix}$$

x_0 is assigned to the class that attains the highest value

SVMs in Action

```
[15]: X = digits.data
      y = digits.target

      X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.5, shuffle=False
      )
```

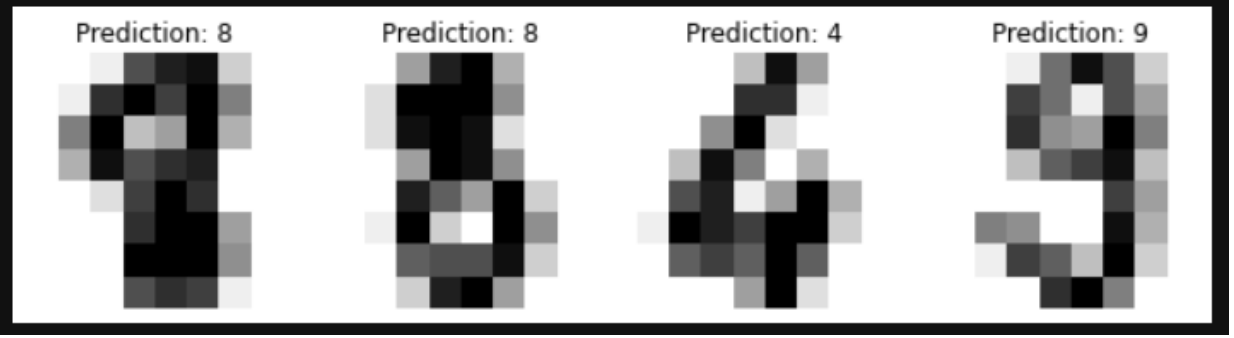
Generating and splitting the dataset

```
[16]: from sklearn import svm

      # Create a classifier: a support vector classifier
      clf = svm.SVC(kernel='rbf', gamma=0.001)

      # Learn the digits on the train subset
      clf.fit(X_train, y_train)

      # Predict the value of the digit on the test subset
      y_pred = clf.predict(X_test)
```

Predicting with Support Vector Machine

SVMs in Action

```
[17]: _, axes = plt.subplots(nrows=1, ncols=4, figsize=(10, 3))
for ax, image, prediction in zip(axes, X_test, y_pred):
    ax.set_axis_off()
    image = image.reshape(8, 8)
    ax.imshow(image, cmap=plt.cm.gray_r, interpolation="nearest")
    ax.set_title(f"Prediction: {prediction}")
```



Showing predictions for the first 4 images

SVMs in Action

```
[18]: print(
          f"Classification report for classifier {clf}:\n"
          f"{classification_report(y_test, y_pred)}\n"
      )
```

```
Classification report for classifier SVC(gamma=0.001):
              precision    recall  f1-score   support

           0       1.00      0.99      0.99        88
           1       0.99      0.97      0.98        91
           2       0.99      0.99      0.99        86
           3       0.98      0.87      0.92        91
           4       0.99      0.96      0.97        92
           5       0.95      0.97      0.96        91
           6       0.99      0.99      0.99        91
           7       0.96      0.99      0.97        89
           8       0.94      1.00      0.97        88
           9       0.93      0.98      0.95        92

    accuracy                           0.97       899
```

Showing scores for each class